Docker Overview and Docker Installation

Docker official website: http://www.docker.com

Docker Hub (registry) official website: https://hub.docker.com

1.1 Docker Overview

Docker is an open-source application container engine project developed in the Go language.

1.1.1 Why Docker Was Created

First, let us consider a few scenarios:

1. The operations engineer deploys a project you developed to a server, then tells you it fails to start. You run it locally and it works fine...
2. A project about to be released becomes unusable because some software has been updated to a newer version...
3. A project involves a huge number of environment components — all kinds of middleware and configuration — and still has to be deployed on multiple servers...

In short, all of these problems come down to the environment. To avoid the various problems caused by environment differences, the best approach is to deploy the project together with all of the environments it requires. For example, if a project involves Redis, MySQL, JDK, Elasticsearch, and other environments, you bundle the entire environment when deploying the JAR package. The question then arises: how can a project be deployed along with its environment?

Docker is the solution to this problem!

1.1.2 Docker's Core Idea



This is the Docker logo — a whale carrying a load of containers. On the whale's back, the containers are isolated from one another, and this is exactly Docker's core idea. For example, when multiple applications previously ran on the same server, there could be conflicts over port usage; now that they are isolated, each application can run independently. In addition, Docker makes the most of the server's capabilities.

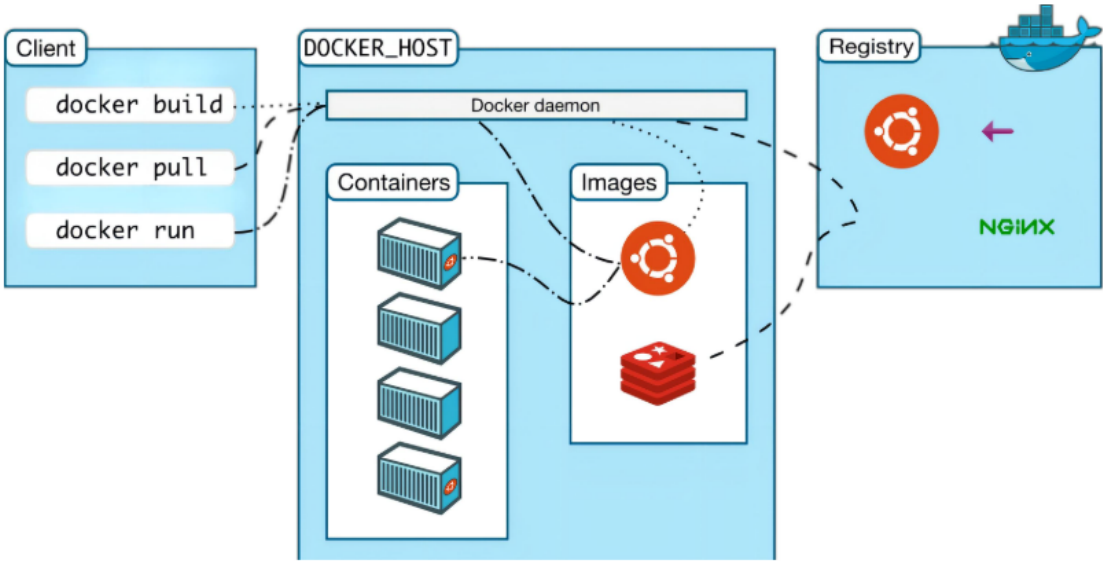
1.1.3 Comparison Between Virtual Machines and Docker

- The Docker daemon can communicate directly with the host operating system and allocate resources to each Docker container. It can also isolate the containers from the host OS and from one another. A virtual machine can take minutes to boot, whereas a Docker container can start within milliseconds. Because there is no bulky guest OS, Docker can save a great deal of disk space and other system resources.
- Virtual machines are better at completely isolating an entire runtime environment. For example, cloud service providers typically use virtual machine technology to isolate different

users. Docker, in contrast, is usually used to isolate different applications, such as the frontend, the backend, and the database.
- Docker containers consume fewer resources than virtual machines and are much faster (to start, stop, create, and delete).

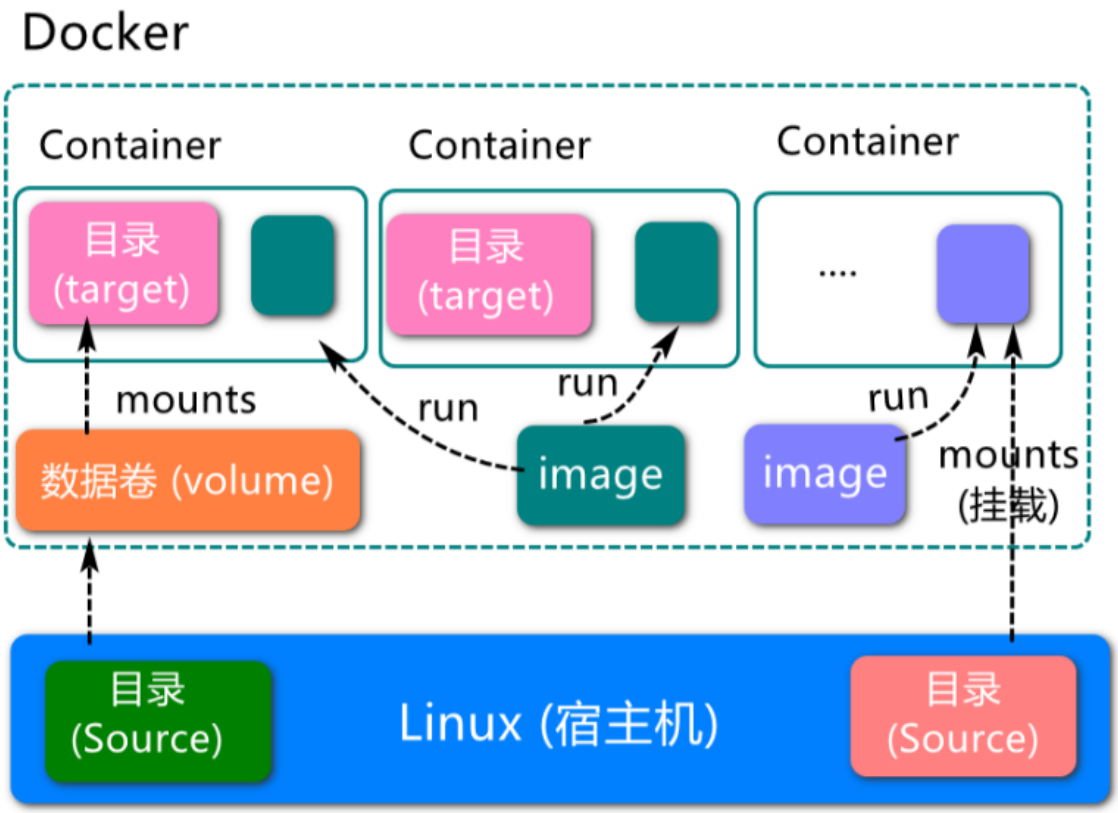
1.1.4 Docker Architecture

Docker uses a client-server architecture. The Docker client communicates with the Docker daemon, which shoulders the heavy work of building, running, and distributing Docker containers. The Docker client and the daemon can run on the same system, or you can connect the Docker client to a remote Docker daemon. The Docker client and the daemon communicate with each other through a UNIX socket or a network interface using the REST API. Another Docker client is Docker Compose, which lets you work with applications consisting of a group of containers.



- The Docker client is the `docker` command used directly after Docker has been installed.
- Docker host is our Docker host machine (that is, the operating system on which Docker is installed).
- The Docker daemon is Docker's background daemon process. It listens for and handles Docker client commands, and manages Docker objects such as images, containers, networks, and volumes.
- The registry is the remote repository from which Docker pulls images. It provides a large number of images for download; after being downloaded, the images are stored in the local image repository (images).
- The images repository is Docker's local image store; you can view the image files with the `docker images` command.

1.1.5 Docker Core Objects



1.1.6 Image, Container, and Repository

Image:

```
A Docker image is a read-only template. An image can be used to create Docker
containers, and one image can create many containers. It is similar to classes
and objects in Java: the class is the image, and the container is the object.
```

Container:

```
Docker uses containers to run one application or a group of applications
independently. A container is a running instance created from an image. It can be
started, begun, stopped, and deleted. Each container is isolated from the others,
providing a secure platform. A container can be regarded as a simplified Linux
environment (including root user privileges, process space, user space, network
space, and so on) together with the applications running inside it. The
definition of a container is almost identical to that of an image – both are a
unified view of a stack of layers – the only difference being that the topmost
layer of a container is readable and writable.
```

Repository:

```
A repository is a place where image files are centrally stored. Repositories come
in two forms: public repositories and private repositories.

The largest public repository is Docker Hub (https://hub.docker.com/), which
stores a huge number of images for users to download. Public repositories in
China include Alibaba Cloud, NetEase Cloud, and others.
```

It is necessary to correctly understand several concepts — the repository, the image, and the container:

- Docker itself is a container runtime carrier, or what might be called a management engine. We package an application and its configuration dependencies into a deliverable runtime environment, and this packaged runtime environment is exactly the image file. Only through this image file can Docker containers be generated. An image file can be seen as a template for containers. Docker generates container instances from the image file. Multiple container instances can be generated from the same image file and run at the same time.
- The container instance generated from an image file is itself also a file, called the image file.
- Each container runs one kind of service. When we need it, we can create a corresponding running instance through the Docker client — that is, our container.
- As for the repository, it is simply a place that holds a collection of images. We can publish images to the repository, and pull them down from the repository whenever needed.

1.1.7 Docker's Working Mechanism

The execution process of `docker pull`:

1. The client sends the command to the Docker daemon.
2. The Docker daemon first checks whether the relevant image exists in the local image store.
3. If the image does not exist locally, it requests the image server and downloads the remote image to the local machine.

The execution process of `docker run`:

1. Check whether the specified image exists locally; if not, pull it from the public repository.
2. Create and start a container from the image.
3. Allocate a file system (a simplified Linux system) and mount a readable and writable layer on top of the read-only image layer.
4. Bridge a virtual interface from the host's configured bridge interface into the container.
5. Assign an IP address to the container from the address pool.
6. Execute the application specified by the user.

1.2 Docker Installation

1. Official installation reference manual: https://docs.docker.com/engine/install/ubuntu/
2. You can use the following command for one-click installation:

```
curl -fsSL https://get.docker.com | bash -s docker --mirror Aliyun
```

⚠ If network errors such as 443 occur during the installation, you may need to access the internet through a VPN.

3. Check the Docker version

```
sudo docker version
```

```
pi@raspberrypi:~ $ sudo docker version
Client: Docker Engine - Community
 Version:           28.2.2
 API version:       1.50
 Go version:        go1.24.3
 Git commit:        e6534b4
 Built:             Fri May 30 12:07:27 2025
 OS/Arch:           linux/arm64
 Context:           default

Server: Docker Engine - Community
 Engine:
  Version:          28.2.2
  API version:      1.50 (minimum version 1.24)
  Go version:       go1.24.3
  Git commit:       45873be
  Built:            Fri May 30 12:07:27 2025
  OS/Arch:          linux/arm64
  Experimental:     false
 containerd:
  Version:          1.7.27
  GitCommit:        05044ec0a9a75232cad458027ca83437aae3f4da
 runc:
  Version:          1.2.5
  GitCommit:        v1.2.5-0-g59923ef
 docker-init:
  Version:          0.19.0
  GitCommit:        de40ad0
pi@raspberrypi:~ $
```

4. Test command

⚠ If network errors such as 443 occur during the installation, you may need to access the internet through a VPN.

```
sudo docker run hello-world
```

If the output is as shown below, Docker has been installed successfully.

```
jetson@ubuntu:~$ sudo docker run hello-world
[sudo] password for jetson:
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
7050e35b49f5: Pull complete
Digest: sha256:4e83453afed1b4fa1a3500525091dbfca6ce1e66903fd4c01ff015dbcb1ba33e
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (arm64v8)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```